Guía del notebook `1_03_1_perceptron_simple`: el perceptrón en Python

Puntos: 3 · **Tiempo:** 15–20 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_1_perceptron_simple.ipynb`
Teoría completa: Guía de la Clase 2, sección B3.

¿Qué vas a aprender?

- A programar las dos piezas del perceptrón: la **combinación lineal** y la **función escalón**.
- A unirlas para hacer una predicción ("pase frontal").

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_1_perceptron_simple.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Combinación lineal:** $h = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$. Multiplica cada entrada por su peso, suma todo y suma el sesgo.
- **Escalón:** devuelve 1 si $h \geq 0$, y 0 si $h < 0$.
- **Perceptrón:** salida = escalón(combinación lineal).

Paso a paso

Celda "Cargamos paquetes"

Carga NumPy. Ejecútala.

Celda `TODO (1 punto)` : `combinacion_lineal(W, X, b)`

Sin usar `np.dot`. Pistas:

1. Empieza con `suma = 0.0`.
2. Recorre los pares (peso, entrada) a la vez con `for w, x in zip(W, X):` y acumula `w * x`.
3. Al final devuelve la suma más `b`.

Celda `TODO (1 punto)` : `escalon(h)`

Debe devolver 1 si $h \geq 0$ y 0 si no, y funcionar tanto con un número como con un arreglo de NumPy.

Pista: la comparación `h >= 0` da `True` / `False`. Conviértela a entero, por ejemplo con `np.asarray(h >= 0).astype(int)`.

Celda "Perceptrón simple"

Ya está hecha: une tus dos funciones. Ejecútala.

Celda "Definamos unos pesos y sesgo"

Usa entradas [0.7, −0.3], pesos [0.1, 0.8] y sesgo −0.1.

✅ **Comprobación:** imprime `Output:` y luego **0**. La cuenta es h = 0.07 − 0.24 − 0.1 = −0.27, que es menor que 0.

Celda `TODO (1 punto)`: pesos para OR

Busca por prueba y error pesos y sesgo que den **OR**: 0 en (0, 0) y 1 en los otros tres casos. Pruébalos con las 4 entradas usando tu función `perceptron`.

✅ **Comprobación:** una solución válida (pesos [1, 1], sesgo −0.5) imprime:

```
(0, 0) -> 0
(0, 1) -> 1
(1, 0) -> 1
(1, 1) -> 1
```

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported operand ... NoneType`	`combinacion_lineal` todavía devuelve `None`	Completa la función y vuelve a ejecutar esa celda
La salida es `True` / `False` en vez de 1/0	No convertiste a entero	Usa `.astype(int)`
Da 1 en lugar de 0 en el ejemplo	Olvidaste sumar el sesgo, o usaste `>` al revés	Revisa `+ b` y la condición `>= 0`